

Database System Tutorial 1

Database TA Team
The University of Auckland



Tutorial Structure

- ❑ **Additional Material:** Brief overview or expansion of key topics
- ❑ **Exercises:** Time to work through problems
- ❑ **Open Q&A:** Discuss any questions

(Slide available on Canvas, Week Overview page)

Differences Between Joins

Features	Cartesian Product	Theta Join	Natural Join
Condition	None	General predicate (e.g., =, <, >)	Implicit equality on matching column names
Duplicate Columns	No removal	No removal	Removes duplicate columns
Symbol	$R \times S$	$R \bowtie_{\theta} S$	$R \bowtie S$

Exercises

- ❑ *branch(branch name, branch city, assets)*
- ❑ *customer (ID, customer name, customer street, customer city)*
- ❑ *loan (loan number, branch name, amount)*
- ❑ *borrower (ID, loan number)*
- ❑ *account (account number, branch name, balance)*
- ❑ *depositor (ID, account number)*

1. Consider this bank database. Assume that branch names and customer names uniquely identify branches and customers, but loans and accounts can be associated with more than one customer.

1.1 What are the appropriate primary keys?

1.2 Given your choice of primary keys, identify appropriate foreign keys.

Exercises

- ❓ *branch(branch name, branch city, assets)*
- ❓ *customer (ID, customer name, customer street, customer city)*
- ❓ *loan (loan number, branch name, amount)*
- ❓ *borrower (ID, loan number)*
- ❓ ✓ *account (account number, branch name, balance)*
- ❓ ✓ *depositor (ID, account number)*

2. Consider this bank database. Give an expression in the relational algebra for each of the following:

select loan number from loan where amount > 10000;

2.1 Find each loan number with a loan amount greater than \$10000.

2.2 Find the ID of each depositor who has an account with a balance greater than \$6000. *select ID from depositor ~~is~~ account where balance > 6000;*

2.3 Find the ID of each depositor who has an account with a balance greater than \$6000 at the “Uptown” branch.

Select ID from depositor ✓ account where balance > 6000
and
branch name = "Vptown";

Exercises

3. Consider the university database as below.

Write the following queries in relational algebra:

3.1 Find the ID and name of each instructor in the Physics department.

$\pi_{ID, name} (\sigma_{dept_name = "Physics"} (instructor))$

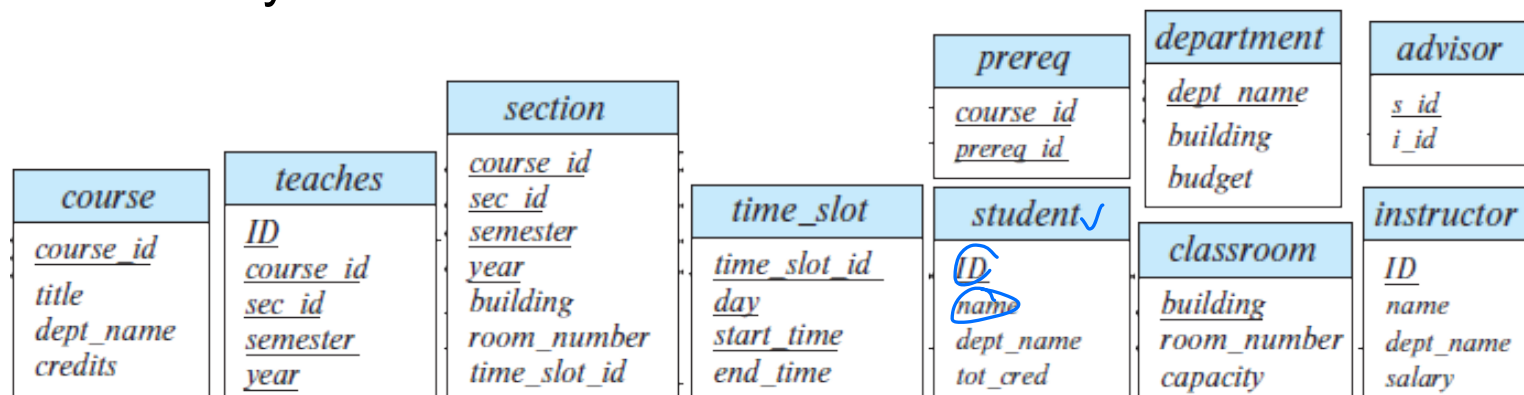
3.2 Find the ID and name of each instructor in a department located in the building "Watson".

$\pi_{ID, name} (\sigma_{building = "Watson"} (instructor \bowtie department))$

3.3 Find the ID and name of each student who has taken at least one course in the "Comp. Sci." department.

3.4 Find the ID and name of each student who has taken at least one course section in the year 2018.

3.5 Find the ID and name of each student who has not taken any course section in the year 2018.



Exercises

4. Consider the university database as below.

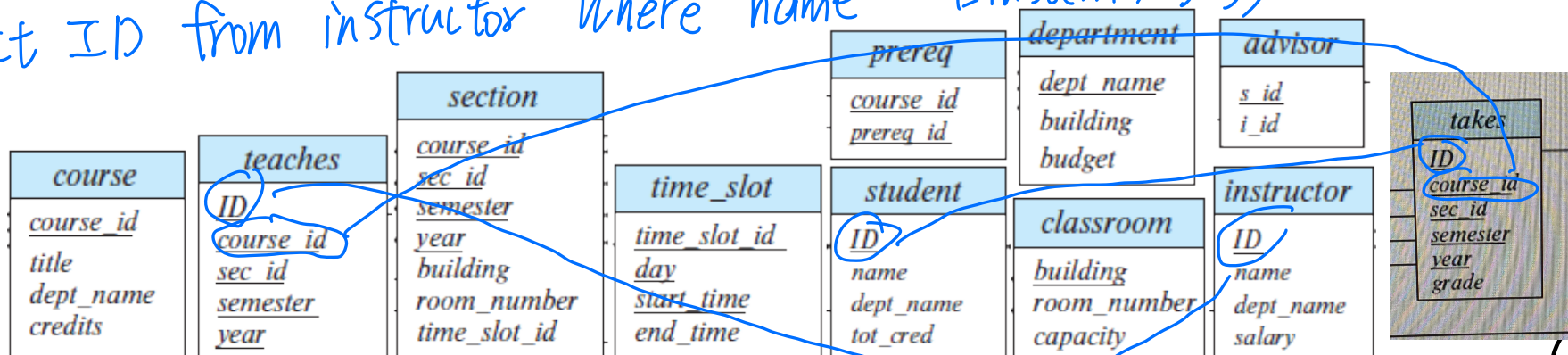
Write the following queries in **SQL**:

4.1 Find the titles of courses in the Comp. Sci. department that have 3 credits.

4.2 Find the IDs of all students who were taught by an instructor named Einstein; make sure there are no duplicates in the result.

4.1. select title from course where dept_name = "Comp. Sci" and credits = 3 ;

4.2. Select distinct ID from Student Where ID = (select ID from takes where course_id = (select course_id from teaches where ID = (select ID from instructor where name = "Einstein"))) ;



Appendix:

University Database

classroom(building, room number, capacity)

department(dept_name, building, budget)

course(course_id, title, dept_name, credits)

instructor(ID, name, dept_name, salary)

section(course_id, sec_id, semester, year, building, room number, time_slot_id)

teaches(ID, course_id, sec_id, semester, year)

student(ID, name, dept_name, tot cred)

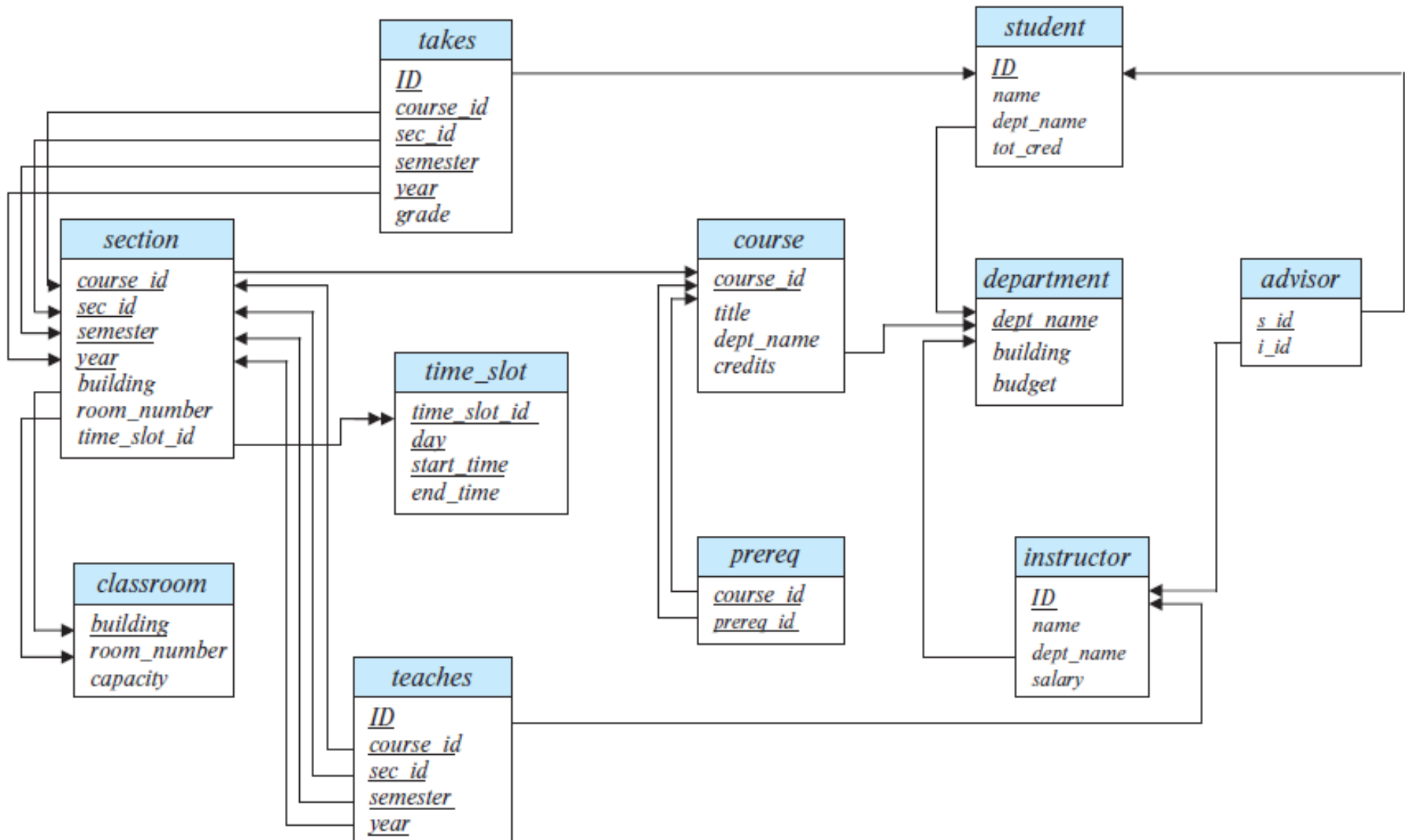
takes(ID, course_id, sec_id, semester, year, grade)

advisor(s_ID, i_ID)

time_slot(time_slot_id, day, start time, end time)

prereq(course_id, prereq_id)

Appendix: University Database



Appendix: University Database



<https://www.db-book.com/university-lab-dir/sqljs.html>

<- Online SQL interpreter

University database loaded



FIN

Any questions?